

THS-II · EMS · Reinforcement Learning Pipeline

v2.1 — Toyota Prius Gen 3 (ZVW30)

PPO Agent · GPS Route Planning · Custom Gymnasium · ONNX / Raspberry Pi HIL

Drive Modes: EV · ECO · NORMAL · PWR (Prius Gen 3 Factory Buttons)

CONFIDENTIAL — All instructions are implementation-ready starting points.

Oussama Chouaibi · EABA | Znazen Khalil · Karoui Wassim · Chouaibi Mohamed Aziz | INSAT 2025/2026

Parameter	Value
Vehicle	Toyota Prius Gen 3 (ZVW30) — THS-II Power-Split Hybrid
RL Algorithm	PPO — Proximal Policy Optimisation (Stable-Baselines3)
Physics Engine	modeling1.py v4 — 2ZR-FXE · PSD · MG1/MG2 · NiMH
Gymnasium	Custom env wrapping THSIIController.step()
Drive Cycles	WLTC Class 3 · FTP-75/UDDS · US06
GPS / Routing	OpenRouteService API · Elevation · Traffic · Segment Classification
Drive Modes	EV · ECO · NORMAL · PWR (4 modes — Prius Gen 3 factory buttons only)
Timeline	3-Day Sprint (Core) + Day 4 GPS/Route + Day 5 CARLA Bonus + Day 6 CAN HIL
HIL Platform	Raspberry Pi 4 — 4 GB RAM [Optional Day 6]
Dashboard	Streamlit web app — live SOC, fuel, GPS telemetry

v2.1 Correction — Drive Mode Alignment: This revision removes the SPORT mode which is NOT present on the Toyota Prius Gen 3 (ZVW30). The Prius Gen 3 provides exactly **three driver-selectable buttons**: **EV MODE**, **ECO MODE**, and **PWR MODE**. The fourth mode, **NORMAL**, is the default state (no button pressed). SPORT mode belongs to other Toyota/Lexus platforms (e.g. Camry XSE, GS hybrid) and was incorrectly carried over from the THS-II generalised documentation. All action spaces, reward functions, CAN frame encodings, and evaluation scripts have been updated accordingly.

Table of Contents

- 1. Introduction & Problem Statement
- 2. State of the Art
- 3. System Architecture & Solution
- 4. System Components & Technologies
- 5. Reinforcement Learning Design
- 6. GPS Route Planning & Road Topology Integration
- 7. DAY 1 · Environment Setup & Physics Verification
- 8. DAY 2 · Gymnasium Env + PPO Training
- 9. DAY 3 · Evaluation, ONNX Export & Dashboard
- 10. DAY 4 [NEW] · GPS Route Integration & Predictive EMS
- 11. DAY 5 [OPTIONAL] · CARLA Live Co-Simulation
- 12. DAY 6 [OPTIONAL] · CAN Bus Hardware-in-the-Loop
- 13. Project Deliverables
- 14. Conclusion & Future Work

1 · Introduction & Problem Statement

The Toyota Hybrid System II (THS-II), deployed in the Prius Gen 3 (ZVW30), uses a planetary power-split device (PSD) that allows the internal combustion engine (ICE), Motor-Generator 1 (MG1), and Motor-Generator 2 (MG2) to operate simultaneously or independently. This architecture makes energy management a complex, multi-dimensional optimisation problem that is poorly served by static rule tables.

The Prius Gen 3 exposes **three physical driver-selectable buttons** on the centre console: **EV MODE**, **ECO MODE**, and **PWR MODE**. The default state when no button is pressed is **NORMAL MODE**. These four states constitute the complete, factory-accurate driver-selectable mode set for this vehicle. No SPORT mode exists on the Gen 3 platform — that feature belongs to other Toyota/Lexus hybrid variants.

This project replaces conventional hand-crafted EMS policies with a Reinforcement Learning agent trained on the high-fidelity modeling1.py physics engine — a production-grade THS-II v4 simulator with real BSFC maps, planetary gear dynamics, MG1/MG2 efficiency lookup tables, a NiMH thermal model, DC-DC converter model, and blended braking logic. The v2.1 pipeline adds GPS-based route segmentation, road topology analysis, elevation and traffic integration, and a dedicated Day 4 for predictive EMS.

1.1 Project Objectives

ID	Objective
OBJ-01	Wrap modeling1.py physics inside a standards-compliant Gymnasium environment
OBJ-02	Train a PPO agent to select optimal driving modes per road segment type
OBJ-03	Minimise total fuel consumption over standardised drive cycles (WLTC, FTP-75, US06)
OBJ-04	Maintain battery SOC within ± 5 % of the 60 % charge-sustaining reference
OBJ-05	Integrate GPS route data — elevation, segment type, traffic — as predictive state features
OBJ-06	Export the trained policy to ONNX; validate inference on Raspberry Pi 4
OBJ-07	Expose results through an interactive Streamlit dashboard with GPS map
OBJ-08	[Optional D5] Integrate RL mode output with live CARLA traffic co-simulation
OBJ-09	[Optional D6] Validate ONNX policy in CAN Bus Hardware-in-the-Loop on RPi 4

1.2 Scope & Constraints

Parameter	Value
Physics Engine	modeling1.py v4 — 2ZR-FXE, PSD, MG1/MG2, NiMH — DO NOT MODIFY
Drive Modes	EV · ECO · NORMAL · PWR (4 discrete actions — Prius Gen 3 factory buttons)
Drive Cycles	WLTC Class 3 · FTP-75/UDDS · US06 (.csv files)
GPS / Route	OpenRouteService API or local OSRM — elevation + traffic labels per segment
Timeline	3 days core + 1 GPS day + 1 optional CARLA + 1 optional CAN HIL
HIL Platform	Raspberry Pi 4 (4 GB RAM) + MCP2515 CAN HAT [Optional D6]
Budget	~€60 hardware (RPi 4 + CAN HAT)

2 · State of the Art

2.1 Rule-Based EMS

Early EMS implementations use deterministic switching rules derived from expert knowledge (charge-depleting / charge-sustaining strategies, thermostat control). They are computationally cheap but cannot account for route topology or traffic context and require manual calibration per vehicle variant.

2.2 Optimal Control Methods

Dynamic Programming (DP) yields globally optimal policies over a known drive cycle but requires full future trip knowledge — making it acausal. Pontryagin's Minimum Principle and ECMS reduce cost but rely on heuristic co-state estimation. MPC adds a rolling prediction horizon at the cost of solving a quadratic programme every timestep.

2.3 Learning-Based EMS — This Work

PPO-based agents have shown 12–15 % fuel reductions on standard cycles with SOC deviations below 3 %. The distinguishing contribution here is threefold: (1) the high-fidelity modeling1.py engine with real lookup tables; (2) GPS-driven predictive state enrichment that gives the agent road topology context before each segment; (3) a low-cost HIL pipeline that exports to ONNX for real-time inference on a Raspberry Pi 4, replicable without proprietary hardware.

2.4 Prius Gen 3 Drive Mode Architecture

The Prius Gen 3 (ZVW30, 2009–2015) provides driver-selectable mode buttons that modify the EMS strategy thresholds inside the HV ECU. Understanding the precise behaviour of each button is critical for accurate RL action space design:

Mode	Activation	Behaviour Summary	EMS Effect
NORMAL	Default (no button)	Balanced ICE/motor split, standard regen	Baseline SOC target 60 %
ECO	ECO button pressed	Throttle map softened, A/C load reduced, lower EV threshold	Enables low ICE load, maximises regen
PWR	PWR button pressed	Aggressive throttle map, higher MG2 boost, earlier ICE engage	Raises SOC target, responsive torque
EV	EV button pressed	Pure electric below 72 km/h, SOC ≥ 45 % required	ICE disabled unless SOC drops or speed exceeds 0

Note: SPORT mode is absent on the Prius Gen 3. SPORT appears on the Camry XSE Hybrid, GS 450h, and other Lexus IS/GS platforms. Any prior documentation referencing SPORT for this vehicle is incorrect and has been removed in this v2.1 revision.

2.5 GPS & Route Planning

Route-aware EMS requires segment-level data: road type, slope, speed limits, and traffic density. Map-matching libraries (OSRM, Valhalla) and elevation APIs (Open-Elevation, SRTM) provide this data. Route segmentation divides a trip into homogeneous chunks (urban / rural / highway) using speed-limit topology and OSM road class tags, enabling the RL agent to receive a predictive lookahead state.

2.6 THS-II vs THS-V

Feature	THS-II (This Project)	THS-V (Future Work)
Generation	Prius Gen 3 (ZVW30)	Prius 60-series
Drive Mode Buttons	EV / ECO / PWR (+ NORMAL default)	EV / ECO / PWR (+ HV default)
Efficiency	Good	Better (+~8 %)
Research maturity	Mature / well-documented	Newer — fewer open datasets
Status	✓ Used (modeling1.py v4)	Future work (FW-04)

3 · Proposed Architecture & Solution

3.1 Phase Overview

Phase	Description	Key Action	Output
Phase 1 — Training (PC)	Gymnasium env wrapping modeling1.py	PPO agent learns mode-switching	SB3 checkpoint saved
Phase 2 — GPS Route (PC)	ORS/OSRM route data → segment features	Predictive state enrichment	GPS-aware episodes
Phase 3 — Evaluation (PC)	SIL validation on all 3 drive cycles	ONNX export	Streamlit dashboard
Phase 4 — CARLA [Opt D5]	RL mode output replaces rule selector	CarlaTHSII Simulation subclass	Live traffic validation
Phase 5 — CAN HIL [Opt D6]	ONNX policy runs on RPi 4	States/modes via CAN Bus 2.0B 500 kbit/s	Embedded validation

3.2 Driving Modes — Prius Gen 3 Factory-Accurate 4-Mode Action Space

The action space is reduced from 5 to **4 modes** to match exactly the Toyota Prius Gen 3 driver-selectable button set. The AUTOMATIC mode remains reserved for the physics engine's internal fallback and is NOT exposed as an RL action.

Action	Mode	Behaviour	Target Segment	GPS Trigger Condition
0	ECO	Conservative regen, softened throttle map, low ICE load	Urban stop/reduction	≤ 15 km/h or traffic > 0.5
1	NORMAL	Balanced ICE/motor torque split, standard regen — default state	Mid/highway	15–80 km/h
2	PWR	Higher ICE output, aggressive MG2 boost, responsive throttle map	Highway/grade	≥ 80 km/h or grade > 4 %
3	EV	Pure electric below 72 km/h, ICE off, SOC ≥ 45 % required	speed urban	Queue / crawl / traffic > 0.7

Note: SPORT mode has been removed entirely. It does not exist as a driver-selectable button on the Prius Gen 3 ZVW30. Previous references to SPORT in action 4 and reward penalties are deprecated.

4 · System Components & Technologies

4.1 The modeling1.py Physics Engine

modeling1.py (THS-II v4) is the powertrain physics core. Its THSIIController class exposes a .step(throttle, brake, speed, grade, dt) method returning a full telemetry dictionary every timestep. The Gymnasium environment wraps this controller directly — no simplified approximations.

Parameter	Value
ICE Model	2ZR-FXE Atkinson 73 kW — BSFC map (RPM × Torque), friction FMEP, coolant thermal
Power Split Device	Planetary K=2.6, dual-mesh efficiency 0.980
MG1 (Sun gear)	42 kW peak, 10 000 rpm limit, 7×7 efficiency table
MG2 (Ring/Output)	60 kW peak, 207 Nm, reduction 2.636×3.267, 7×7 efficiency table
HV Battery	NiMH 201.6 V / 6.5 Ah, RC thermal model, OCV-SOC curve, SOC 40–80 %
DC-DC Converter	500 V HV bus, boost η =0.972, buck η =0.968
Blended Braking	Hydraulic fade-in above 0.3 g decel, regen priority
EV Mode Cap	72 km/h (20 m/s), SOC ≥ 45 % required — matches Prius Gen 3 factory spec
ICE Anti-Hunt	Dual threshold P _{on} =4 kW / P _{off} =1.5 kW + 10 s minimum warmup hold

4.2 Custom Gymnasium Environment

The environment wraps THSIIController. The state space exposes 8 variables (5 powertrain + 3 GPS-derived); the action space maps to the 4 Prius Gen 3 DriveMode values. The reward penalises fuel consumption and SOC deviation while rewarding regenerative energy capture.

Variable	Unit	Source	Description
velocity	m/s	speed arg / drive cycle	Current vehicle speed (normalised ÷ 30)
SOC	[0,1]	state.soc_pct / 100	Battery state-of-charge
road_grade	rad	GPS elevation / grade arg	Segment slope (normalised ÷ 0.3)
segment_type	{0,1,2}	GPS road class	0=urban, 1=rural, 2=highway (÷ 2)
acceleration	m/s²	Δ speed/dt	Instantaneous acceleration (÷ 5)
gps_lookahead_grade	rad	Next-segment elevation	Preview of upcoming slope (÷ 0.3)
gps_traffic_density	[0,1]	ORS / real-time traffic API	Traffic congestion level (0=free, 1=jam)
gps_dist_to_next_seg	m	Route segment boundary	Distance to segment type change (÷ 1000)

Reward Function

Term	Expression	Purpose
Fuel penalty	- out["fuel_rate_gs"]	Minimise instantaneous fuel rate
SOC penalty	- 10.0 × (soc - 0.60)²	Keep SOC at 60 % reference
Regen bonus	+ 0.5 × max(0, -out["p_batt_kw"])	Reward regenerative braking capture
Grade anticipation	+ 0.3 × (EV on flat) — 0.2 × (EV on grade > 5 %)	Penalise EV on steep uphill (GPS-aware)

Note: The SPORT overuse penalty (previously " $- 2.0 \times \text{SPORT_steps} / \text{total_steps}$ ") has been removed. SPORT mode does not exist on Prius Gen 3. The reward function now uses exactly 4 terms.

4.3 PPO Hyperparameters

Parameter	Value
Algorithm	PPO — Proximal Policy Optimisation
Policy Network	MlpPolicy — 2×64 hidden layers, tanh activation
Learning Rate	3×10^{-4} with linear decay
n_steps	2048
Batch Size	64
n_epochs	10
Clip Range	0.2
GAE Lambda	0.95
Discount γ	0.99
Total Timesteps	500 000 (SIL) / 800 000 (GPS-enriched episodes recommended)
Train Time	~10 min on laptop CPU (~6 min with GPU)
Observation Space	Box(8,) — 5 powertrain + 3 GPS features
Action Space	Discrete(4) — EV / ECO / NORMAL / PWR [was Discrete(5) in v2.0 — SPORT removed]

4.4 Technology Stack

Technology	Version	Purpose
Python	3.11	Core language
modeling1.py (v4)	—	High-fidelity THS-II powertrain physics — DO NOT MODIFY
Gymnasium	0.29	RL environment API wrapper
Stable-Baselines3	2.x	PPO training framework
PyTorch	2.x	Neural network backend
OpenRouteService API	—	GPS route data, elevation profile, segment classification
OSRM (local)	—	Optional offline routing engine
Open-Elevation API	—	SRTM-based elevation data along GPS routes
ONNX Runtime	1.17	Embedded inference on RPi 4 [Optional Day 6]
python-can	4.x	CAN Bus communication [Optional Day 6]
NumPy / Pandas	—	Numerical computation & drive cycle data I/O
Matplotlib / Plotly	—	Result visualisation & publication figures
Folium / Leaflet	—	GPS route map rendering in Streamlit
Streamlit	1.x	Interactive web dashboard
CARLA	0.9.x	Live traffic co-simulation [Optional Day 5]

5 · Reinforcement Learning Design

5.1 MDP Formulation

Parameter	Value
State S	8-dimensional vector: [velocity, SOC, road_grade, segment_type, acceleration, gps_lookahead_grade, gps_traffic_density, gps_dir]
Action A	Discrete {0:EV, 1:ECO, 2:NORMAL, 3:PWR} — mirrors Prius Gen 3 DriveMode enum
Reward R	$-\text{fuel_rate_gs} - \lambda \cdot (\text{SOC} - 0.60)^2 + \gamma_{\text{regen}} \cdot \text{regen_energy} + \text{gps_anticipation_bonus}$
Transition T	THSIIController.step() — full powertrain dynamics, no approximation
Episode	One full drive cycle (WLTC: 1800 s · FTP-75: ~2474 s · US06: 596 s) or one GPS route
Discount γ	0.99 — long-horizon fuel and SOC shaping

5.2 Drive Cycle & GPS Segmentation Strategy

Condition	Segment	Expected Mode	GPS Context
$v < 15$ km/h	Urban (segment_type = 0)	ECO or EV preferred	High traffic density expected
$15 \leq v < 80$ km/h	Rural (segment_type = 1)	NORMAL preferred	Mixed — GPS grade drives sub-mode
$v \geq 80$ km/h	Highway (segment_type = 2)	PWR or NORMAL preferred	Pre-emptive PWR from grade lookahead
GPS: grade > 4 %	Uphill (any segment)	PWR	Pre-emptive from lookahead
GPS: grade < -2 %	Downhill (any segment)	ECO (max regen)	Lookahead triggers early ECO
GPS: traffic > 0.7	Stop-go urban	EV preferred	Queue anticipation

5.3 Reward Tuning Guidelines

Symptom	Likely Cause	Fix Action
SOC drops below 45 %	lambda_soc too low	Increase lambda_soc: 10 → 15 → 20
Agent always selects EV	Regen bonus too high	Reduce gamma_regen: 0.5 → 0.2
Reward flat after 100k steps	Learning rate too high	Decay LR: 3e-4 → 1e-4
Fuel not improving vs baseline	Action not applied to EMS	Verify DriveMode is set in step()
Episode terminates too early	SOC cutoff too strict	Check the soc < 0.40 condition
Mode oscillation every few steps	Insufficient hysteresis	Add mode hold counter (≥ 3 steps)
GPS features ignored	Feature scaling wrong	Verify all GPS obs normalised to [0,1]
EV selected at high speed	Missing EV cap enforcement	Enforce EV→ECO fallback above 72 km/h in step()

5.4 Evaluation Metrics

Metric	Definition	Target
Fuel Savings (%)	RL vs rule-based baseline on test cycle	> 5 %
SOC Deviation	RMSE of SOC from 60 % across full episode	< ± 5 %
Mode Distribution	Histogram of mode selections per segment type	Qualitative

GPS Anticipation Rate	% of grade events where mode set before segment	> 60 %
Inference Latency (PC)	ONNX sess.run() time on PC	< 2 ms
Inference Latency (RPI)	ONNX sess.run() time on RPi 4 [Optional HIL]	< 10 ms
CAN Reliability	Dropped / corrupt frames [Optional HIL]	< 0.1 %
RAM Footprint	Peak RAM on RPi during HIL [Optional HIL]	< 200 MB
CPU Usage	Average CPU load on RPi [Optional HIL]	< 30 %

6 · GPS Route Planning & Road Topology Integration

This chapter defines the GPS pipeline that enriches the RL state vector with predictive route knowledge. The pipeline is independent of modeling1.py and can be swapped between online (API-based) and offline (OSRM/local) modes without touching the physics engine.

6.1 Architecture Overview

- Origin + Destination → ORS/OSRM API → Route GeoJSON (waypoints + road class)
- Route GeoJSON + Open-Elevation API → Elevation profile per waypoint (SRTM 30 m resolution)
- Elevation profile → Gradient computation ($\text{grade_rad} = \text{atan}(\Delta h / \Delta d)$ per segment)
- Road class tags (OSM: motorway / primary / residential) → segment_type labels
- ORS traffic extension or HERE Traffic API → traffic_density per segment
- Pre-trip: full route processed → list of RouteSegment objects saved to route_cache.json
- At runtime: THSEnv reads route_cache.json and injects GPS features each step

6.2 Route Segmentation Strategy

OSM Road Class	Segment Type	Speed Range	Preferred Mode
motorway / motorway_link	highway (type=2)	≥ 80 km/h	PWR or NORMAL
primary / secondary / trunk	rural (type=1)	15–80 km/h	NORMAL
residential / tertiary	urban (type=0)	< 15 km/h	ECO or EV
living_street / pedestrian	urban crawl (type=0)	< 5 km/h	EV mandatory
Any + grade > 4 %	uphill override	any	PWR
Any + grade < -2 %	downhill override	any	ECO (max regen)

6.3 Elevation & Gradient Computation

- $\text{grade_rad}[i] = \text{atan2}(\text{elevation}[i+1] - \text{elevation}[i], \text{haversine_distance}(\text{waypoint}[i], \text{waypoint}[i+1]))$
- Haversine distance: $d = 2R \cdot \arcsin(\sqrt{(\sin^2(\Delta\phi/2) + \cos\phi_1 \cdot \cos\phi_2 \cdot \sin^2(\Delta\lambda/2))})$
- Positive grade → uphill → PWR anticipation
- Negative grade → downhill → ECO / max regen anticipation
- Lookahead window: 500 m ahead of current vehicle position → gps_lookahead_grade in state
- Savitzky-Golay filter applied (window=11, poly=2) to smooth noisy SRTM data

6.4 Traffic & Environmental Data Integration

Traffic Density [0,1]	Recommended Mode Bias
Free flow (< 0.3)	NORMAL or PWR — maintain speed, ICE-efficient operating point
Moderate (0.3–0.6)	ECO — lower EV threshold, pre-charge for stop-go
Heavy (0.6–0.8)	EV priority — frequent stops favour pure-electric short bursts
Jam (> 0.8)	EV mandatory if SOC ≥ 45 %, else ECO with maximum regen on decel

6.5 GPS Feature Injection into THSEnv

The THSEnv.__init__() method accepts an optional route_cache path. When provided, each step() call queries the pre-computed RouteSegment list by matching current distance_travelled to segment boundaries, and injects GPS features into the 8-dimensional observation. For standard drive-cycle training without GPS, the three GPS dimensions default to zero.

```
current_seg = find_segment_by_distance(self.distance_m, self.route_segments)
next_seg = route_segments[current_seg.id + 1] # lookahead
obs[5] = np.clip(next_seg.grade_rad / 0.3, -1, 1) # gps_lookahead_grade
obs[6] = current_seg.traffic_density # gps_traffic_density
obs[7] = (next_seg.start_dist_m - self.distance_m) / 1000.0 # gps_dist_to_next_seg
self.distance_m += speed_ms * dt # integrate vehicle position
```

DAY 1

Environment Setup & Physics Verification

Foundation · All subsequent days depend on this

Day 1 is non-negotiable. Every downstream task assumes a verified, fully-functional modeling1.py standalone run and a properly configured Python environment. Do not proceed to Day 2 until all checkpoints below are green.

Part 1A · Python Stack Installation

- Create a dedicated virtual environment (Python 3.11 recommended) and activate it.
- Install RL and simulation stack: stable-baselines3[extra], gymnasium==0.29, torch (CPU build unless CUDA), onnxruntime, tensorboard.
- Install data/visualisation stack: numpy, pandas, matplotlib, plotly, streamlit, folium, streamlit-folium.
- Install GPS stack: requests, openrouteservice, geopy, scipy (for Savitzky-Golay filter), shapely.
- Install optional: python-can (for CAN HIL), carla (only if CARLA is installed).
- Freeze requirements into requirements.txt immediately after installation.
- Verify all imports in a Python REPL — any ImportError must be resolved before continuing.

Part 1B · Project Directory Structure

Path	Description
env/	Gymnasium environment module (ths_env.py + drive cycle CSVs)
env/drive_cycles/	WLTC.csv, FTP75.csv, US06.csv — speed_ms column required
gps/	GPS pipeline scripts: route_fetcher.py, elevation.py, segmenter.py, route_cache.json
gps/cache/	Pre-computed route JSON files for reuse across training runs
training/	PPO training script, baseline comparator, export script
eval/	SIL evaluation, plotting, and metrics scripts
models/	Saved SB3 checkpoints and ONNX model output
hil/	CAN Bus PC-side and RPi controller scripts [Optional Day 6]
carla/	CARLA RL bridge subclass [Optional Day 5]
app/	Streamlit dashboard script
tb_logs/	TensorBoard log directory (auto-created)
modeling1.py	THS-II physics engine — place at project root, DO NOT MODIFY

Part 1C · Physics Verification

- Run modeling1.py --standalone. Confirm KPI values print and it exits without errors.
- Run --standalone --ftp75. All 2474 timesteps must complete.
- Run --standalone --plot. Verify 4×2 subplot PNG saved to /tmp/, all 8 subplots populated.
- Open ths2_kpis.csv. Confirm 28 columns, zero NaN in SOC or fuel_rate_gs columns.
- Inspect THSIIController class: note DriveMode enum — confirm it contains ECO, NORMAL, PWR, EV. Verify SPORT is absent (Prius Gen 3 platform). AUTOMATIC is reserved and not exposed.
- Document standalone fuel total for WLTC and FTP-75 — this is the physics-engine baseline for Day 3.

Part 1D · Drive Cycle Data Preparation

- Download WLTC Class 3 (UNECE source). Convert to CSV: speed_ms column, 1800 rows.
- Download FTP-75/UDDS (US EPA). Same format: speed_ms column, ~2474 rows.
- Download US06 (US EPA). Same format: speed_ms column, 596 rows.

- Optionally add road_grade_rad column (zeros for flat cycle) — GPS pipeline will overwrite on Day 4.
- Place all three files in env/drive_cycles/ and verify row counts before continuing.

Part 1E · Per-Mode KPI Baseline Recording

Before any RL training, run modeling1.py in fixed-mode operation across all three drive cycles and all **four DriveMode values (EV, ECO, NORMAL, PWR)**. This creates a mode-locked KPI table serving as the definitive comparison baseline for Day 3.

KPI	Unit	Expression / Source
total_fuel_g	g	$\Sigma \text{fuel_rate_gs} \times \text{dt}$
fuel_per_km	g/km	$\text{total_fuel_g} / \text{cycle_distance_km}$
soc_final	%	state.soc_pct at last step
soc_rmse	%	RMSE(soc_pct, 60.0) over episode
soc_min	%	min(soc_pct) over episode
regen_total_j	J	$\Sigma \max(0, -\text{p_batt_kw}) \times 1000 \times \text{dt}$
ice_on_fraction	—	Fraction of steps where p_ice_kw > 0
ev_fraction	—	Fraction of steps in pure EV operation
early_termination	bool	True if SOC < 0.40 triggered early stop

Note: Expected CSV: eval/per_mode_kpis.csv — 12 rows (4 modes × 3 cycles). Schema: cycle, mode, total_fuel_g, fuel_per_km, soc_final, soc_rmse, soc_min, regen_total_j, ice_on_fraction, ev_fraction, episode_steps, early_termination

Day 1 — Checkpoints

- ✓ python modeling1.py --standalone → prints KPIs, zero errors
- ✓ python modeling1.py --standalone --ftp75 → 2474 steps complete
- ✓ --plot flag → 4×2 subplot PNG saved, all 8 subplots populated
- ✓ ths2_kpis.csv → 28 columns, no NaN in SOC or fuel_rate_gs
- ✓ WLTC.csv / FTP75.csv / US06.csv present in env/drive_cycles/ with correct row counts
- ✓ All Python imports (SB3, Gymnasium, PyTorch, ONNX, Streamlit, ORS, folium) succeed
- ✓ Directory structure created with gps/ and gps/cache/ present
- ✓ [CORRECTED] eval/per_mode_kpis.csv — 12 rows (4 modes × 3 cycles), no NaN — SPORT removed
- ✓ eval/figures/per_mode_fuel_bar.png saved at 300 dpi — 4 modes visible for 3 cycles
- ✓ eval/figures/per_mode_soc_traces.png — SOC traces for all 12 combinations plotted
- ✓ NORMAL mode WLTC fuel_g documented as primary Day 3 RL target

DAY 2

Gymnasium Env + PPO Training

Core RL pipeline · Reward tuning · Baseline comparison

Part 2A · Implementing env/th_s_env.py

- `__init__`: Accept cycle ("WLTC", "FTP75", "US06"), `dt=0.1`, and optional `route_cache=None`. Define `observation_space` as `Box(8,)` with `low=-1`, `high=1`, `dtype=float32`. Define `action_space` as `Discrete(4)` [4 modes, not 5 — SPORT removed].
- `reset()`: Instantiate `THSIIController`(`init_drive_mode=DriveMode.ECO`) each call. Load drive cycle CSV. If `route_cache` is not `None`, load `RouteSegment` list from JSON. Reset `idx=0`, `distance_m=0.0`, `self.speed=0.0`.
- `step()`: Receive integer action (0–3). Map to `DriveMode` enum: 0→EV, 1→ECO, 2→NORMAL, 3→PWR. Set `ems.state.selector_mode`. Compute throttle/brake from speed delta. Call `ems.step()`. Compute GPS features if `route_cache` loaded. Compute reward. Advance `idx`.
- `_obs()`: Build the 8-element normalised observation. GPS dims default to 0.0 if no `route_cache`.
- Termination: `done=True` when `idx` reaches end of cycle OR `soc < 0.40`. `Truncated=False` always.
- Guard against CARLA/pygame at module level with `try/except ImportError`.

Part 2B · Environment Validation with check_env

- Run `check_env(THSEnv("WLTC"))` before any training. Resolve all warnings.
- Run a manual episode with random actions (actions 0–3 only). Verify `obs[5:8]` are 0.0 without `route_cache`.
- Run again with a sample `route_cache`. Verify `obs[5:8]` contain non-zero values and change at segment boundaries.
- Confirm `done` flag at step 1800 for WLTC. Print `DriveMode` each step — confirm only EV/ECO/NORMAL/PWR appear.

Part 2C · Rule-Based Baseline (training/baseline_rule.py)

- Instantiate `THSEnv("WLTC")` and run with rule: EV if $v < 5$ km/h AND $soc \geq 0.45$; ECO if $v < 15$ km/h; NORMAL if $v < 80$ km/h; PWR otherwise.
- Record total fuel (sum of `fuel_rate_gs × dt`) in grams and SOC trajectory.
- Save these numbers — PPO must beat this by $\geq 5\%$ on Day 3.

Part 2D · PPO Training (training/train_ppo.py)

- Instantiate `THSEnv("WLTC")` for training and a separate instance for evaluation. Never share instances.
- Configure PPO: `learning_rate=3e-4`, `n_steps=2048`, `batch_size=64`, `n_epochs=10`, `gamma=0.99`, `gae_lambda=0.95`, `clip_range=0.2`, `tensorboard_log="./tb_logs/"`.
- Add `EvalCallback`: `eval_freq=50_000`, `best_model_save_path="./models/"`.
- Run `model.learn(total_timesteps=500_000)`. Monitor `ep_rew_mean` in TensorBoard.
- Save final model as `models/th_s_agent_final`. Best checkpoint auto-saved as `models/best_model.zip`.

Day 2 — Checkpoints

- ✓ `check_env(THSEnv("WLTC"))` → zero warnings or errors
- ✓ Random episode completes 1800 steps, `done` flag raised cleanly
- ✓ `action_space` is `Discrete(4)` — confirmed by `env.action_space`
- ✓ Only EV / ECO / NORMAL / PWR actions sampled — no action 4 / SPORT
- ✓ GPS dims (`obs[5:8]`) are 0.0 without `route_cache`; non-zero with `route_cache`
- ✓ Baseline rule total fuel recorded in grams
- ✓ PPO training starts and TensorBoard `ep_rew_mean` rising by step 100k
- ✓ `models/best_model.zip` exists after `EvalCallback` fires at 50k steps

DAY 3

Evaluation, ONNX Export & Dashboard

SIL validation · ONNX · Streamlit · Final report figures

Part 3A · Software-in-the-Loop Evaluation

- Load models/best_model.zip. Run deterministic=True evaluation on WLTC, FTP-75, US06. Run each ≥ 3 times and average.
- Record per cycle: total fuel (g), final SOC, SOC RMSE from 60 %, episode return, mode counts per segment.
- Compare RL fuel vs rule-based baseline AND vs per-mode KPI table (Part 1E). Fuel savings vs NORMAL must exceed 5 % on WLTC.
- Generate publication-quality plots (300 dpi): SOC trajectory, cumulative fuel, mode histogram by segment, reward curve.
- Generate fuel bar chart: 3 groups (WLTC, FTP-75, US06), bars for EV / ECO / NORMAL / PWR / Rule-Based / RL PPO / RL+GPS. Save to eval/figures/.

Part 3B · ONNX Export

- Load models/best_model.zip via PPO.load(). Access underlying policy network.
- Create dummy input shape (1, 8) — matches 8-dimensional GPS-enriched observation.
- Export with torch.onnx.export: opset_version=17, input_names=["obs"], output_names=["action_logits"]. Save to models/th_s_policy.onnx.
- Verify file (~180–220 KB — 4-action output head is slightly smaller than 5-action v2.0).
- Benchmark: 1000 sequential sess.run() calls on PC. Target < 2 ms per call.

Part 3C · Streamlit Dashboard

- Sidebar: drive cycle selector, agent mode (RL PPO / Rule-based / Random), optional GPS route JSON upload.
- "Run Episode" button: executes one deterministic episode, stores per-step telemetry in st.session_state.
- SOC trajectory chart with colour bands for EV / ECO / NORMAL / PWR selections.
- Cumulative fuel chart: bars for EV / ECO / NORMAL / PWR (from per_mode_kpis.csv), rule-based, RL PPO, RL+GPS.
- Mode distribution donut chart: 4 slices (EV / ECO / NORMAL / PWR) with % labels.
- GPS map panel (Folium): if route JSON uploaded, show route polyline coloured by recommended mode per segment.
- Summary metrics table: total fuel (g), fuel savings vs NORMAL (%), SOC RMSE, episode duration.

Day 3 — Checkpoints

- ✓ RL fuel savings on WLTC > 5 % vs NORMAL mode baseline — confirmed in SIL evaluation
- ✓ SOC RMSE < ± 5 % of 60 % across all three drive cycles
- ✓ models/th_s_policy.onnx exists (~180–220 KB), ONNX Runtime inference validated on 8-dim obs
- ✓ ONNX output has 4 logits (not 5) — action_space Discrete(4) confirmed
- ✓ ONNX inference latency < 2 ms per call on PC
- ✓ eval/figures/ contains SOC trace, fuel bar chart (7 bars: 4 modes + rule + PPO + GPS), mode histogram
- ✓ Streamlit dashboard launches and GPS map panel renders when route JSON provided
- ✓ Git tag v2.1-sil committed with all deliverables

DAY 4 [NEW]

GPS Route Integration & Predictive EMS

Route segmentation · Elevation · Traffic · Pre-trip mode planning

Part 4A · GPS Route Fetcher (gps/route_fetcher.py)

- Register for a free OpenRouteService (ORS) API key at openrouteservice.org.
- Implement `fetch_route(origin_ll, destination_ll, api_key)`: call ORS directions endpoint.
- Implement `get_elevation_profile waypoints)`: call Open-Elevation API in batches of 100 points.
- Implement `compute_grades(waypoints, elevations)`: apply Savitzky-Golay filter (`window=11, poly=2`).
- Implement `classify_segments()`: group waypoints by road class into `RouteSegment` objects.
- Save output to `gps/cache/.json` (MD5 of origin/dest coords) to prevent redundant API calls.

Part 4B · Traffic Data Integration

- If using ORS: compute `congestion_ratio = duration / duration_typical` per segment. Map: ratio 1.0 → 0.0, ratio ≥ 2.5 → 1.0.
- If using HERE Traffic API: `density = 1 - (speed / freeflow_speed)` per waypoint.
- For offline testing: set `traffic_density=0.0` for highway, 0.5 for urban as default.

Part 4C · GPS-Enriched Training Episodes

- Create 5 representative synthetic routes: flat urban, hilly rural, steep highway climb, mixed, downhill suburban.
- Create `GPSWrappedEnv` that randomly selects one of the 5 routes each episode `reset()`.
- Fine-tune from Day 2 best checkpoint: 300 000 additional timesteps. Save as `models/th_s_agent_gps.zip`.
- Export new ONNX: `models/th_s_policy_gps.onnx` — verify 4-logit output head.

Part 4D · Pre-Trip Mode Sequence Generator

- Load `models/th_s_agent_gps.zip`. For each `RouteSegment`, build mock observation from segment features.
- Call `model.predict(obs, deterministic=True)` to get recommended mode (0=EV, 1=ECO, 2=NORMAL, 3=PWR).
- Output list: (`segment_id, start_km, end_km, recommended_mode, grade_rad, traffic_density`).
- Save to `gps/pre_trip_plan.json` and print human-readable summary.

Day 4 — Checkpoints

- ✓ `gps/route_fetcher.py` fetches ORS route and elevation — `route_cache` JSON saved
- ✓ Gradients computed and smoothed — `grade_rad` values visually correct
- ✓ Traffic density present in route cache JSON for all segments
- ✓ `THSEnv` with `route_cache` — `obs[5:8]` non-zero and change at segment boundaries
- ✓ GPS-fine-tuned model `th_s_agent_gps.zip` saved with `Discrete(4)` action space
- ✓ `th_s_policy_gps.onnx` has 4-logit output — verified with ONNX Runtime
- ✓ Pre-trip plan JSON generated with EV/ECO/NORMAL/PWR recommendation per segment
- ✓ GPS anticipation rate > 60 % on validation run
- ✓ Streamlit GPS map panel renders route with mode colour-coding

DAY 5 [OPTIONAL]

CARLA Live Co-Simulation

Optional bonus — requires CARLA 0.9.x installed on host

Part 5A · Understanding CarlaTHSIISimulation

- Read CarlaTHSIISimulation class carefully. Identify `_select_drive_mode(speed, soc, grade, seg)`.
- Note how the base class calls `_select_drive_mode()` each tick and uses the returned `DriveMode`.
- Confirm that importing `modeling1.py` with CARLA unavailable raises `ImportError` — guard with `try/except`.

Part 5B · Building the RL Bridge (`carla/carla_rl_bridge.py`)

- Create `RLBridgeSimulation` subclass inheriting from `CarlaTHSIISimulation`.
- In `__init__`, call `super().__init__()` then load `PPO.load("models/th_s_agent_gps.zip")`.
- Override `_select_drive_mode()`: build 8-element obs, call `policy.predict()`, return `DriveMode` enum for action 0=EV, 1=ECO, 2=NORMAL, 3=PWR.
- Log each tick: timestep, `DriveMode`, SOC, speed, CARLA location → `carla/carla_rl_log.csv`.

Day 5 — Optional Checkpoints

- [Optional] CARLA server connects, vehicle spawns in Town03
- [Optional] `_select_drive_mode()` maps actions 0–3 to EV/ECO/NORMAL/PWR only
- [Optional] `carla_rl_log.csv` populated with mode, SOC, speed, location columns
- [Optional] No Python exceptions during a full 5-minute CARLA run

DAY 6 [OPTIONAL]

CAN Bus Hardware-in-the-Loop

Optional HIL — requires RPi 4 + MCP2515 CAN HAT

Part 6A · Hardware Assembly

- Mount MCP2515 CAN HAT on RPi 4. Connect PC USB-CAN adapter via twisted-pair. Place 120 Ω terminators.
- Flash Raspberry Pi OS 64-bit (Bookworm). Enable SPI via raspi-config.
- Add overlay: `dtoverlay=mcp2515-can0,oscillator=8000000,interrupt=25` to `/boot/firmware/config.txt`.
- Loopback test: `cansend` on PC → `candump` on RPi. Confirm correct frame content.

Part 6B · CAN Protocol — Frame Definitions

Frame 0x100 — Vehicle State (PC → RPi) · DLC: 8 bytes · 100 ms cycle

Bytes	Signal	Type	Encoding / Notes
0–1	velocity	uint16	$v \times 100$ [cm/s] — 0.01 m/s resolution
2	SOC	uint8	$SOC \times 100$ [%] — 1 % resolution
3	road_grade	int8	$grade \times 50$ [°] — 0.02° resolution
4	segment_type	uint8	0=urban / 1=rural / 2=highway
5	acceleration	int8	$a \times 10$ [dm/s ²] — 0.1 m/s ² resolution
6	frame_counter	uint8	Rolling 0–255
7	checksum	uint8	XOR bytes 0–6

Frame 0x200 — Mode Decision (RPi → PC) · DLC: 4 bytes

Bytes	Signal	Type	Encoding / Notes
0	mode_action	uint8	0=EV / 1=ECO / 2=NORMAL / 3=PWR [SPORT=4 removed]
1	latency_ms	uint8	ONNX inference time in ms
2	frame_counter	uint8	Echo of received 0x100 counter
3	checksum	uint8	XOR bytes 0–2

Note: The mode_action byte now encodes only 4 values (0–3). Value 4 (previously SPORT) is deprecated and must never appear in the 0x200 frame. Any RPi controller code referencing action 4 must be updated.

Part 6C · HIL Loop & Validation

- `pc_side_can.py`: encode 5 powertrain dims into 0x100 frame, send at 10 Hz, receive 0x200 mode decision.
- `rpi_controller.py`: pre-load GPS route cache, reconstruct 8-dim obs from CAN + GPS lookup, run ONNX inference, encode 0x200.
- Run full WLTC (1800 steps $\times$ 100 ms = 180 s real time). Monitor for CAN errors.
- Verify mean latency < 5 ms, max < 10 ms. CAN error rate < 0.1 %.
- Monitor RPi with `htop`: peak RAM < 200 MB, average CPU < 30 %.

Day 6 — Optional Checkpoints

- [Optional] CAN loopback test passes
- [Optional] `ths_policy_gps.onnx` benchmark on RPi < 5 ms per call (8-dim input)

- [Optional] 0x200 mode_action byte encodes only values 0–3 (EV/ECO/NORMAL/PWR)
- [Optional] Full WLTC HIL episode completes 1800 steps without CAN error
- [Optional] Mean HIL latency < 5 ms, max < 10 ms
- [Optional] CAN frame error rate < 0.1 %
- [Optional] RPi peak RAM < 200 MB, CPU < 30 %

13 · Project Deliverables

ID	Path	Description	Status
SW-01	env/ths_env.py	Gymnasium env (8-dim obs, 4 actions — Discrete(4))	Mandatory
SW-02	modeling1.py	THS-II v4 physics engine — DO NOT MODIFY	Mandatory
SW-03	env/drive_cycles/*.csv	WLTC, FTP-75, US06 drive cycle data	Mandatory
SW-04	training/train_ppo.py	PPO training script with EvalCallback	Mandatory
SW-05	training/baseline_rule.py	Rule-based EMS comparator (4-mode)	Mandatory
SW-06	training/export_onnx.py	PyTorch policy → ONNX (8-dim in, 4-logit out)	Mandatory
SW-07	eval/sil_evaluate.py	SIL validation on all 3 cycles + plots	Mandatory
SW-07b	eval/per_mode_baseline.py	Per-mode KPI baseline (4 modes × 3 cycles)	Mandatory
SW-08	app/streamlit_dashboard.py	Dashboard with GPS map panel (4-mode donut)	Mandatory
SW-09	models/best_model.zip	Best SB3 PPO checkpoint	Mandatory
SW-10	models/ths_policy.onnx	ONNX model (4-logit output)	Mandatory
SW-10b	eval/per_mode_kpis.csv	Per-mode KPI table (12 rows: 4 modes × 3 cycles)	Mandatory
SW-11	gps/route_fetcher.py	ORS route + elevation fetcher	Day 4
SW-12	gps/traffic.py	Traffic density integration	Day 4
SW-13	gps/pre_trip_planner.py	Pre-trip mode sequence generator	Day 4
SW-14	models/ths_agent_gps.zip	GPS fine-tuned SB3 checkpoint	Day 4
SW-15	models/ths_policy_gps.onnx	GPS-aware ONNX model (8-dim, 4-logit)	Day 4
SW-16	carla/carla_rl_bridge.py	CARLA ↔ RL bridge (4-mode)	Optional D5
SW-17	hil/pc_side_can.py	PC CAN state transmitter (4-mode 0x200)	Optional D6
SW-18	hil/rpi_controller.py	RPI ONNX inference + CAN responder	Optional D6
SW-19	eval/hil_evaluate.py	HIL validation with latency logging	Optional D6

14 · Conclusion & Future Work

14.1 Summary

This v2.1 guide delivers a complete, reproducible pipeline for learning-based Energy Management in a Toyota THS-II hybrid vehicle — specifically the **Prius Gen 3 (ZVW30)**. The core contribution is a PPO agent trained on standardised drive cycles using the high-fidelity modeling1.py physics engine with a factory-accurate **4-mode DriveMode action space (EV / ECO / NORMAL / PWR)** — exactly matching the three physical buttons plus the default state on the Prius Gen 3 centre console. The v2.1 revision corrects the erroneous inclusion of SPORT mode and aligns all action spaces, reward functions, CAN frame encodings, and evaluation scripts accordingly.

The v2.1 additions include a GPS route planning pipeline (Day 4) enriching the agent's state with road topology, elevation gradients, and traffic density — enabling predictive, route-aware mode selection before each segment. A per-mode KPI baseline (Part 1E) provides a physics-engine-grounded comparison reference for all four drive modes across all three test cycles.

14.2 Expected Results

Parameter	Value
Fuel savings (SIL, no GPS)	5–15 % vs. NORMAL fixed-mode baseline on WLTC cycle
Fuel savings (GPS-enriched)	Additional 2–5 % from predictive mode anticipation
SOC deviation	< ±5 % RMSE from 60 % across all test cycles
GPS anticipation rate	> 60 % of grade transitions pre-selected correctly
ONNX latency (PC)	< 2 ms per inference call (8-dim input, 4-logit output)
ONNX latency (RPI 4)	< 5 ms per inference call [Optional D6]
CAN frame error rate	< 0.1 % over 1800-step WLTC HIL episode
Dashboard	10 Hz replay + GPS Folium map with EV/ECO/NORMAL/PWR colour-coding

14.3 Future Work

ID	Topic	Description	Effort
FW-01	Full CARLA Emergent Traffic	Replace prescribed cycles with dynamic CARLA scenarios + GPS	High
FW-02	Multi-Objective Reward	Add comfort (jerk) and emissions (NOx) to reward function	Medium
FW-03	Real-Time Google Maps	Live Maps API → segment type prediction before trip start	Medium
FW-04	THS-V Migration	Port modeling1.py physics to THS-V (Prius Gen 5) parameters	High
FW-05	dSPACE HIL Upgrade	Automotive-grade HIL certification platform	High
FW-06	SAC / TD3 Comparison	Benchmark off-policy algorithms for sample efficiency	Low
FW-07	Online Adaptation	Fine-tune policy on-device after deployment	High
FW-08	Federated Fleet Learning	Federated RL across multiple vehicles — shared policy backbone	Research
FW-09	Hierarchical RL	High-level GPS planner + low-level torque controller	Research